React.js

1. Introduction to React.js

THEORY EXERCISE

Question 1: What is React.js? How is it different from other JavaScript frameworks and libraries?

Ans.

React.js is an open-source JavaScript library developed by Facebook for building user interfaces, especially single-page applications (SPAs). It helps developers create fast, interactive, and reusable UI components. React mainly focuses on the view layer of an application and uses a component-based architecture to build modern web applications.

React is different from other JavaScript frameworks and libraries because it uses the Virtual DOM to improve performance and update only the necessary parts of a webpage instead of reloading the entire page. React is also highly flexible and can easily integrate with other libraries and APIs.

React applications are easier to maintain because the code is divided into reusable components. It is widely used for creating dynamic and responsive web applications.

Difference Between React and Other Frameworks/Libraries

React.js	Other Frameworks
React is a library for UI development	Frameworks provide complete solutions
Uses component-based architecture	Some frameworks use template-based architecture
Uses Virtual DOM for faster performance	Many frameworks directly update the DOM
Flexible and easy to integrate	Some frameworks are more rigid

---

Question 2: Explain the core principles of React such as the Virtual DOM and component-based architecture.

Ans.

Virtual DOM

The Virtual DOM is a lightweight copy of the real DOM. React updates the Virtual DOM first and compares it with the previous version. Only changed elements are updated in the real DOM, which improves performance.

Component-Based Architecture

React applications are built using components. Each component is a reusable and independent piece of UI.

Advantages:

- Reusable code
- Easy maintenance
- Better organization
- Faster development

Question 3: What are the advantages of using React.js in web development?

Ans.

React.js provides many advantages in modern web development. It is one of the most popular JavaScript libraries because of its speed, flexibility, and reusable component structure.

Advantages of React.js:

- Fast performance using Virtual DOM
- Reusable components reduce code duplication
- Easy to learn and develop applications
- Better code organization and maintenance
- Supports single-page applications
- Strong community and developer support
- Easy debugging with React Developer Tools
- SEO-friendly with server-side rendering
- Improves user experience with faster UI updates
- Can be used for both web and mobile applications through React Native

These features make React.js a powerful and efficient library for building scalable applications.

LAB EXERCISE

Task:

- Set up a new React.js project using create-react-app.
- Create a basic component that displays "Hello, React!" on the web page.

Ans. `npx create-react-app`

`myapp cd myapp npm start`

App.js

```
function App() {
return (
    <div>
      <h1>Hello, React!</h1>
    </div>
  );
}
export default App;
```

---

2. JSX (JavaScript XML)

THEORY EXERCISE

Question 1: What is JSX in React.js? Why is it used?

Ans.

JSX stands for JavaScript XML. It is a syntax extension used in React.js that allows developers to write HTML-like code inside JavaScript. JSX makes the UI code easier to understand and write because it looks similar to HTML.

JSX is used in React because it simplifies the process of creating user interfaces. It allows developers to combine JavaScript logic and HTML structure in a single file. JSX is not directly understood by browsers, so it is converted into regular JavaScript using Babel.

Advantages of JSX:

- Easier to read and write
- Improves code readability
- Helps create dynamic web pages
- Allows embedding JavaScript expressions using curly braces `{}`
- Makes UI development faster and simpler

JSX is used because:

- It makes code easier to read.
- It simplifies UI development.
- It allows combining HTML and JavaScript together.

Question 2: How is JSX different from regular JavaScript? Can you write JavaScript inside JSX?

Ans.

JSX allows writing HTML-like syntax inside JavaScript. Regular JavaScript does not support HTML syntax directly.

Yes, JavaScript expressions can be written inside JSX using curly braces `{}`.

Example:

```
const name = "Ayush";

function App() {
  return <h1>Hello {name}</h1>;
}
```

Question 3: Discuss the importance of using curly braces {} in JSX expressions.

Ans.

Curly braces `{}` in JSX are used to insert JavaScript expressions inside HTML-like code.

They are used for:

- Variables
- Functions
- Calculations
- Conditional rendering Example:

```
const age = 20;

<h1>Age: {age}</h1>
```

LAB EXERCISE

Task:

Create a React component that renders:

- A heading with the text "Welcome to JSX".
- A paragraph explaining JSX with dynamic data.

Ans.

```
function App() {
```

```
  const topic = "JSX allows HTML inside JavaScript";
   return
(
    <div>
      <h1>Welcome to JSX</h1>
      <p>{topic}</p>
    </div>
  );
}
export default App;
```

---

3. Components (Functional & Class Components)

THEORY EXERCISE

Question 1: What are components in React? Explain the difference between functional components and class components.

Ans.

Components are the reusable building blocks of a React application. They divide the user interface into smaller and independent parts, making applications easier to develop and maintain. Each component can contain its own structure, logic, and styling.

React mainly provides two types of components:

- Functional Components
- Class Components

Functional components are simple JavaScript functions that return JSX. They are easier to write and commonly used in modern React applications.

Class components are ES6 classes that extend `React.Component`. They can use lifecycle methods and state management using `this.state`.

Functional components are preferred today because they are simpler, faster, and support hooks.

Functional Components	Class Components
Simple JavaScript functions	ES6 classes
Use hooks for state management	Use lifecycle methods
Easier and shorter syntax	More complex syntax
Faster and commonly used	Older approach

Question 2: How do you pass data to a component using props?

Ans.

Props are used to pass data from one component to another.

Example:

```
function Greeting(props) {
  return <h1>Hello {props.name}</h1>;
}

<Greeting name="Ayush" />
```

Question 3: What is the role of render() in class components?

Ans.

The `render()` method is used in class components to return JSX that should be displayed on the screen.

Example:

```
class App extends React.Component {
render() {
    return <h1>Hello React</h1>;
  }
}
```

LAB EXERCISE

Task 1:

Create a functional component Greeting.

Ans.

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}
export default Greeting;
```

Task 2:

Create a class component WelcomeMessage.

Ans.

```
import React, { Component } from "react";

class WelcomeMessage extends Component {
render() {
    return <h1>Welcome to React!</h1>;
  }
}
export default WelcomeMessage;
```

---

4. Props and State

THEORY EXERCISE

Question 1: What are props in React.js? How are props different from state?

Ans.

Props in React.js are used to pass data from one component to another component. Props stand for “properties” and help make components reusable and dynamic. They are passed from a parent component to a child component.

State is used to store and manage data inside a component. Unlike props, state can be updated and changed during the execution of the program.

Props and state are both important concepts in React, but they serve different purposes. Props are read-only, while state is mutable and controlled inside the component.

Props	State
Read-only	Mutable
Passed from parent component	Managed inside component
Cannot be modified directly	Can be updated

Question 2: Explain the concept of state in React.

Ans.

State is an object used to store dynamic data in a component. When state changes, the component re-renders automatically.

Question 3: Why is this.setState() used in class components?

Ans.

`this.setState()` is used to update the state in class components. It tells React to re-render the component with updated data.

LAB EXERCISE

Task 1:

Create UserCard component.

Ans.

```
function UserCard(props) {
return (
    <div>
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
<p>Location: {props.location}</p>
    </div>
  );
}
export default UserCard;
```

Task 2:

Create Counter component.

Ans.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
   return
(
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
export default Counter;
```

---

5. Handling Events in React

THEORY EXERCISE

Question 1: How are events handled in React compared to vanilla JavaScript?

Ans.

React uses synthetic events instead of direct DOM events. Synthetic events provide consistent behavior across browsers.

Question 2: What are common event handlers in React?

Ans.

Common event handlers:

- onClick
- onChange
- onSubmit Example:

```
<button onClick={handleClick}>Click</button>
```

Question 3: Why do you need to bind event handlers in class components?

Ans.

Binding is required so that `this` refers to the correct component instance.

LAB EXERCISE

Task 1:

Button click event.

Ans.

```
import { useState } from "react";

function App() {
  const [text, setText] = useState("Not Clicked");
   return
(
<div>
```

```
<h1>{text}
</h1>
      <button onClick={() => setText("Clicked!")}>
        Click
      </button>
    </div>
  );
}
export default App;
```

Task 2:

Dynamic input display.

Ans.

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");
   return (
<div>       <input
type="text"
        onChange={(e) => setName(e.target.value)}
      />

      <h1>{name}</h1>
    </div>
  );
}
export default App;
```

---

6. Conditional Rendering

THEORY EXERCISE

Question 1: What is conditional rendering in React?

Ans.

Conditional rendering in React means displaying different UI elements or components based on specific conditions. It allows developers to show or hide content dynamically according to the application's state, user actions, or other logical conditions. React supports conditional rendering using methods such as if-else statements, ternary operators, and logical AND (`&&`) operators. This feature helps create interactive and user-friendly applications by controlling what should appear on the screen at different times.

Question 2: Explain if-else, ternary operator, and && in JSX.

Ans.

- if-else is used for multiple conditions.
- Ternary operator is used for short conditions.
- && renders content only if condition is true.

Example:

```
{isLogin ? <h1>Welcome</h1> : <h1>Please Login</h1>}
```

LAB EXERCISE

Task 1:

Login/logout button.

Ans.

```
import { useState } from "react";

function App() {
  const [login, setLogin] = useState(false);
   return (
<div>       {
login ?
        <button onClick={() => setLogin(false)}>Logout</button>
        :
        <button onClick={() => setLogin(true)}>Login</button>
}
    </div>
  );
}
export default App;
```

Task 2:

Voting eligibility.

Ans.

```
function App() {
const age = 20;
   return
(
<div>
      {
        age >= 18 ?
        <h1>You are eligible to vote</h1>
```

```
        :
        <h1>You are not eligible to vote</h1>
      }
    </div>
  );
}
export default App;
```

---

7. Lists and Keys

THEORY EXERCISE

Question 1: How do you render a list in React?

Ans.

Lists are rendered using the `map()` function.

Keys are important because they help React identify which items changed.

Question 2: What are keys in React?

Ans.

Keys are unique identifiers used while rendering lists.

Without keys, React may update components incorrectly and reduce performance.

LAB EXERCISE

Task 1:

Fruit list.

Ans.

```
function App() {
  const fruits = ["Apple", "Banana", "Mango"];
   return
(     <ul>
      {
        fruits.map((fruit, index) => (
<li key={index}>{fruit}</li>
        ))
```

```
      }
    </ul>
  );
}
export default App;
```

Task 2:

User list with keys.

Ans.

```
function App() {
const users = [
    { id: 1, name: "Ayush" },
    { id: 2, name: "Udit" }
  ];
return (
<ul>
{
        users.map(user => (
          <li key={user.id}>{user.name}</li>
        ))
      }
    </ul>
  );
}
export default App;
```

---

8. Forms in React

THEORY EXERCISE

Question 1: How do you handle forms in React?

Ans.

Forms in React are handled using controlled components where form data is controlled using state.

Question 2: Difference between controlled and uncontrolled components.

Ans.

Controlled Components	Uncontrolled Components
Managed using state	Managed using DOM

Easy validation	Less control
Recommended in React	Less commonly used

LAB EXERCISE

Task 1:

Form using state.

Ans.

```
import { useState } from "react";

function App() {
  const [form, setForm] = useState({
name: "",     email: "",
password: ""
  });
  const handleChange = (e) => {
setForm({       ...form,
      [e.target.name]: e.target.value
    });
  };    const handleSubmit =
(e) => {
    e.preventDefault();
console.log(form);
  };
return (
    <form onSubmit={handleSubmit}>
      <input type="text" name="name" onChange={handleChange} />
      <input type="email" name="email" onChange={handleChange} />
      <input type="password" name="password" onChange={handleChange} />
      <button type="submit">Submit</button>
    </form>
  );
}
export default App;
```

---

9. Lifecycle Methods

THEORY EXERCISE

Question 1: What are lifecycle methods?

Ans.

Lifecycle methods are special methods in React class components that run during different phases of a component.

Phases:

- Mounting
- Updating
- Unmounting

Question 2: Explain componentDidMount(), componentDidUpdate(), and componentWillUnmount().

Ans.

- componentDidMount() runs after component loads.
- componentDidUpdate() runs after updates.
- componentWillUnmount() runs before component removal.

LAB EXERCISE

Task 1:

Fetch API data.

Ans.

```
import React, { Component } from "react";
 class App extends Component
{   componentDidMount() {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
.then(data => console.log(data));
  }
render() {
    return <h1>Data Fetched</h1>;
  }
}
export default App;
```

---

10. Hooks

THEORY EXERCISE

Question 1: What are React hooks?

Ans.

Hooks are functions that allow functional components to use state and lifecycle features.

useState() manages state. useEffect()
handles side effects.

Question 2: What problems did hooks solve?

Ans.

Hooks reduced complexity in class components and improved code reuse.

Question 3: What is useReducer?

Ans.

useReducer is a hook used for complex state management.

Question 4: Purpose of useCallback & useMemo.

Ans.

- useCallback memoizes functions.
- useMemo memoizes computed values.

Question 5: Difference between useCallback & useMemo.

Ans.

useCallback	useMemo
Returns memoized function	Returns memoized value
Used for functions	Used for calculations

Question 6: What is useRef?

Ans.

useRef is a hook used to directly access DOM elements without re-rendering.

LAB EXERCISE

Task 1:

Counter using useState.

Ans.

```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);
   return
(
    <div>
      <h1>{count}</h1>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrement
      </button>
    </div>
  );
}
export default App;
```

---

11. Routing in React

THEORY EXERCISE

Question 1: What is React Router?

Ans.

React Router is a library used for routing in React single-page applications.

Question 2: Difference between BrowserRouter, Route, Link, and Switch.

Ans.

- BrowserRouter enables routing.
- Route displays components.
- Link navigates between pages.
- Switch renders only one route.

LAB EXERCISE

Task 1:

Basic routing.

Ans.

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function Home() {
  return <h1>Home Page</h1>;
}
function About() {
  return <h1>About Page</h1>;
}  function App() {
return (
<BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

---

12. React – JSON-server and Firebase

THEORY EXERCISE

Question 1: What are RESTful web services?

Ans.

RESTful web services are APIs that use HTTP methods like GET, POST, PUT, DELETE for communication.

Question 2: What is Json-Server?

Ans.

Json-Server is a tool used to create a fake REST API using a JSON file.

Question 3: How do you fetch data from Json-server API?

Ans.

Data is fetched using fetch() or axios().

Example:

```
fetch("http://localhost:3000/users")
  .then(res => res.json())
  .then(data => console.log(data));
```

Question 4: What is Firebase?

Ans.

Firebase is a Backend-as-a-Service platform developed by Google.

Features:

- Authentication
- Realtime Database
- Hosting
- Cloud Storage

Question 5: Importance of error handling and loading states.

Error handling improves reliability and loading states improve user experience.

Ans.

13. Context API

THEORY EXERCISE

Question 1: What is Context API?

Ans.

Context API is used to share global state across multiple components without prop drilling.

Question 2: Explain createContext() and useContext().

Ans.

- createContext() creates context.
- useContext() accesses shared data.

---

14. State Management (Redux, Redux Toolkit, Recoil)

THEORY EXERCISE

Question 1: What is Redux?

Ans.

Redux is a state management library used in React applications.

Core concepts:

- Actions
- Reducers
- Store

Question 2: How does Recoil simplify state management?

Ans.

Recoil provides easier state management with less boilerplate compared to Redux.

LAB EXERCISE

Task 1:

Redux counter app.

Ans.

```
import { createStore } from "redux";

const reducer = (state = 0, action) => {
switch(action.type) {     case
"INCREMENT":       return state + 1;
     case "DECREMENT":
return state - 1;

default:
      return state;
  }
};
const store = createStore(reducer);
```

Task 2:

Todo list using Recoil.

Ans.

```
import { atom } from "recoil";
 export const todoState =
atom({   key: "todoState",
default: []
});
```

Task 3:

CRUD using Redux Toolkit.

```
import { createSlice } from "@reduxjs/toolkit";

const userSlice = createSlice({
name: "users",   initialState:
[],
   reducers:
{
    addUser: (state, action) => {
state.push(action.payload);
    },
    deleteUser: (state, action) => {
      return state.filter(user => user.id !== action.payload);
```

Ans.

```
    }
  }
});
export const { addUser, deleteUser } = userSlice.actions;

export default userSlice.reducer;
```